Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

Contents

Executive Summary — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 5

- The problem . . . 5
- The solution . . . 5
- How it works . . . 5
- Why it matters . . . 6
- Important boundaries . . . 6

Solution Overview — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 6

- What this is . . . 6
- How the technology works (the mental model) . . . 7
- The standard deployment model . . . 7
- Scope — what this solution covers . . . 8
- Out of scope (handled elsewhere) . . . 8
- Authentication and connections used . . . 8

Requirements — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 9

- R1. Telemetry is already flowing (a connected workspace) . . . 9
- R2. Roles to author rules, watchlists, and read incidents . . . 9
- R3. The Content hub solutions for the detections in scope . . . 10
- R4. A named enrichment data set for watchlists . . . 10
- R5. An agreed detection priority and false-positive tolerance . . . 10
- R6. A version-control repository for detection-as-code (optional but recommended) . . . 11
- R7. The PowerShell toolchain . . . 11

Global Requirements — Sentinel Solutions 11

- G1. An Azure subscription . . . 12
- G2. A Log Analytics workspace . . . 12
- G3. Microsoft Sentinel enabled on the workspace . . . 12
- G4. Administrative roles (least privilege) . . . 12
- G5. Ingestion and retention cost plan . . . 13
- G6. Diagnostic settings capability on each source . . . 13
- G7. Change control & rollback posture . . . 13

Implementation — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 14

- Object naming (used verbatim throughout) . . . 14
- Phase 0 — Assess (map coverage, rank gaps) . . . 14
- Phase 1 — Configure (watchlists, then enable a small high-value set) . . . 15

Phase 2 — Pilot (let it run, measure the noise) . . . 16
Phase 3 — Tune (make the alerts trustworthy) . . . 16
Phase 4 — Validate . . . 16
Phase 5 — Expand (promote, export as code, repeat the loop) . . . 17
Rollback (for any step) . . . 17
Testing & Validation — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 17
Gate 1 — Telemetry the rules need is present . . . 17
Gate 2 — Baseline coverage map exists and gaps are ranked . . . 18
Gate 3 — Enrichment watchlists imported and referenced . . . 18
Gate 4 — Pilot rules enabled with entity mappings, in low-impact posture . . . 18
Gate 5 — Alert volume and false-positive rate measured . . . 18
Gate 6 — False-positive rate within tolerance (trust gate) . . . 19
Gate 7 — Targeted ATT&CK; gaps are now covered . . . 19
Gate 8 — Rules exported as code . . . 19
Gate 9 — Promotion and rollback are documented . . . 19
Troubleshooting — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 20
An enabled rule never fires . . . 20
A rule fires constantly (false-positive flood) . . . 20
A rule template does not appear under Rule templates . . . 20
The MITRE ATT&CK; coverage view looks empty or wrong . . . 20
A watchlist reference errors in a rule . . . 21
NRT rule will not save / behaves like a scheduled rule . . . 21
False-positive rate looks fine but real incidents are missed . . . 21
Promoted rule floods incidents after enabling incident creation . . . 21
PowerShell / export errors . . . 22
Best Practices — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 22
Detect against a map, not a checklist . . . 22
Tune for trust — a noisy detection is a liability . . . 22
Pilot before you trust . . . 23
Enrich for precision . . . 23
Treat detections as code . . . 23
Run it as a program . . . 23
Where to go next . . . 23
Deliverables Summary — Microsoft Sentinel Analytics Rules & MITRE ATT&CK; Detection Coverage 24
Documentation . . . 24

PowerShell tools (5) 25
Named objects (verbatim — used across docs, scripts, and tests) 26
What this solution deliberately does NOT do 26
How it is delivered 27
Appendix A — PowerShell Scripts 28

Executive Summary — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

The problem

A SIEM that ingests logs but has no tuned detections is an **expensive log bucket**. Two failure modes are common, and organizations usually suffer both at once:

• **Blind spots.** No one can say which attacker techniques the SIEM would actually catch. Coverage is assumed, not measured — so a whole class of intrusion (say, credential theft or privilege escalation) may raise no alert at all, and nobody knows until an incident makes it obvious.

• **Alert fatigue.** The detections that *are* on fire off constantly with false positives. Analysts learn to close them without looking, and a real alert drowns in the noise. A detection nobody trusts is worse than no detection — it consumes the team and provides false comfort.

The root cause is the same in both cases: rules were turned on (or left off) without a **map of what matters** and without **tuning to the environment**.

The solution

This solution turns raw telemetry into **detections the SOC can trust**, mapped to a shared language of attacker behavior — the **MITRE ATT&CK** matrix. It:

• **Enables the right analytics rules** — Microsoft-provided **rule templates**, **scheduled** KQL rules, and **near-real-time (NRT)** rules — chosen against a coverage map, not at random.

• **Maps coverage to MITRE ATT&CK and does gap analysis** — so you can *see*, tactic by tactic and technique by technique, what you detect and where you are blind, and close the highest-risk gaps first.

• **Tunes thresholds and entity mappings to cut false positives** — turning noisy rules into precise ones with suppression, allow-lists, and environment-specific thresholds, so alerts are believable.

• **Adds watchlists and enrichment** — VIP users, authorized admin IPs, terminated staff, high-value assets — so detections carry context and fire smarter.

• **Exports detections as code** — every rule versioned in a repository (detection-as-code) so your detection logic is reviewable, repeatable, and portable across workspaces.

How it works

We follow the same repeatable model every solution in this program uses — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. We first **map current coverage** and rank gaps by risk, then enable a small, high-value set of rules and run them in a **low-impact pilot** (rules produce alerts you triage but do not yet trigger automation), **tune** them against real data until the false-positive rate is acceptable, **validate** that coverage improved and noise dropped, and only then **expand** across the ATT&CK matrix. Anything disruptive is piloted before it is trusted.

The solution ships as **self-guided, click-by-click documentation plus five PowerShell tools** (Az / Microsoft.Graph + KQL) that discover current rules and templates, build the ATT&CK coverage/gap matrix, report per-rule alert and **false-positive** rates for tuning, validate the detection set against pass/fail gates, and export the rules as code — all **read-only or export-only**, with **no credentials in code**.

Why it matters

- **Coverage you can see** — an ATT&CK map that shows exactly what you detect and where you are blind,

so investment goes to the gaps that matter.

- **Alerts the team believes** — tuned thresholds and enrichment mean fewer false positives and faster,

more confident triage.

- **Detections as durable assets** — versioned, reviewable, portable detection-as-code, not clicks in a

portal that no one can reproduce.

- **A fraction of consulting cost** — the same disciplined method a detection engineer would use,

delivered as a repeatable, tested runbook.

Important boundaries

This solution **tunes and maps detections; it is not a guarantee of coverage** — MITRE ATT&CK is a model, and a technique shown as "covered" means a rule targets it, not that every variant will be caught. It **assumes the telemetry is already flowing**: rules can only detect what the workspace ingests, so it depends on a connected, cost-aware foundation (the companion **Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion** solution). It **does not** decide your organization's **risk tolerance, alerting priorities, or which techniques are in scope** — those are security-leadership determinations; the solution provides the technical method and the measured coverage to inform them. Microsoft rule templates, the NRT rule set, and ATT&CK technique mappings **evolve and can differ by cloud (Commercial vs Government)** — *verify current availability in your tenant*.

Solution Overview — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

What this is

A complete, self-guided method for turning the telemetry already in your Microsoft Sentinel workspace into **tuned, trustworthy detections mapped to the MITRE ATT&CK matrix** — with explicit gap analysis, false-positive reduction, enrichment, and **detection-as-code** so your detection logic is a versioned, portable asset rather than a pile of portal clicks.

It is the **detection** layer of a Sentinel program. It assumes a connected, cost-aware workspace (the companion *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution) and builds the analytics rules on top of it.

How the technology works (the mental model)

• **Analytics rules.** The detections in Sentinel. The main types:

• **Scheduled rules** — a **KQL** query that runs on a schedule (e.g. every 5–60 minutes), looks back over a window, and raises an **alert** (and optionally an **incident**) when the query returns results over a threshold. The workhorse of custom detection.

• **Near-real-time (NRT) rules** — a special scheduled rule that runs about once a minute for low-latency detection of high-priority events. Limited in number and query shape (*verify current limits in your tenant*).

• **Microsoft Security / Fusion / ML rules** — rules that surface alerts from Microsoft security products or Microsoft's correlation/ML. You enable rather than author these.

• **Rule templates** — Microsoft- and community-provided detections (installed via the Content hub) you enable and then tune, instead of writing every rule from scratch.

• **MITRE ATT&CK mapping.** ATT&CK is a matrix of adversary **tactics** (the "why" — e.g. *Credential Access*, *Privilege Escalation*) and **techniques** (the "how" — e.g. *T1110 Brute Force*). Each analytics rule is tagged with the tactics/techniques it detects. Aggregating those tags across all enabled rules produces a **coverage map**; the untagged cells are your **gaps**.

• **Tuning (threshold & entity).** A rule fires too often when its **threshold** is too low or it lacks **allow-lists/suppression** for known-benign activity. Tuning adjusts the query, threshold, and grouping, and adds **entity mappings** (user, host, IP) so alerts are correlated into meaningful incidents and known-good sources are excluded.

• **Watchlists & enrichment. Watchlists** are reference tables (VIP users, authorized admin IPs, terminated staff, high-value assets) you join to in rule queries — so a rule can, for example, fire *only* when a sensitive account is involved, or *suppress* when the source is a sanctioned scanner.

• **Automation rules.** Rules that act on incidents (assign, change severity, suppress duplicates, trigger a playbook). Used here mainly for **suppression/tuning**, not full SOAR.

• **Detection-as-code.** Exporting rules to a version-controlled format (ARM/JSON, or YAML via Sentinel **Repositories**) so detections are reviewed, diffed, and deployed like software.

The standard deployment model

Every solution in this program follows the same five stages:

1. **Assess.** Inventory current enabled rules and available templates; build the **ATT&CK coverage map** and rank gaps by risk (run `SENTRULE-Discovery.ps1`, `SENTRULE-Coverage.ps1`).

1. **Configure.** Create the enrichment **watchlists**, enable a small high-value rule set against the top-risk gaps, and set entity mappings — naming every object.

1. **Pilot.** Run the new rules in a **low-impact** posture — they produce alerts you triage, but

incident creation/automation is held back — so you can measure noise before it reaches the queue.

1. **Validate.** Confirm coverage improved and the **false-positive rate** is acceptable per rule; tune thresholds/suppression until it is (run `SENTRULE-Report.ps1`, `SENTRULE-Validate.ps1`).

1. **Expand.** Turn the tuned rules to full incident creation, then repeat the map→enable→tune loop across the rest of the ATT&CK matrix, exporting each tuned rule as code (`SENTRULE-Export.ps1`).

Scope — what this solution covers

- **Rule enablement** — scheduled rules, NRT rules, and Microsoft-provided rule templates.
- **MITRE ATT&CK coverage mapping and gap analysis** — a measured map and a risk-ranked gap list.
- **Threshold and entity tuning** — cutting false positives with thresholds, grouping, suppression, and allow-lists.
- **Watchlists and enrichment** — reference data joined into detections for context and precision.
- **Detection-as-code export** — versioning and reuse of tuned rules.

Out of scope (handled elsewhere)

- **Standing up the workspace and connectors / ingestion** — the companion *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution (this detection layer depends on it).
- **Full SOAR / playbook automation** beyond suppression/tuning automation rules — a separate Sentinel automation solution.
- **Threat-hunting content and notebooks** beyond what supports tuning.
- **The determination of risk tolerance and alerting priorities** — which techniques matter most, and what false-positive rate is acceptable, are security-leadership decisions; this solution measures and implements against the priorities you set.

Authentication and connections used

- **Microsoft Sentinel** (Azure portal or Defender portal) for rule authoring, watchlists, and MITRE view.
- **PowerShell** with the **Az** modules (`Az.Accounts`, `Az.SecurityInsights`, `Az.OperationalInsights`) for rule/template/watchlist discovery, coverage/gap analysis, per-rule reporting, validation, and detection-as-code export; **Microsoft.Graph** where identity context is needed.
- **KQL** run against the workspace (via `Invoke-AzOperationalInsightsQuery`) for false-positive/alert

reporting.

• All tools authenticate **interactively** with an MFA-capable admin; **no credentials are stored in code**. Government-cloud tenants pass the matching environment — *verify endpoints for your cloud*.

Requirements — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

These are the **solution-specific** prerequisites for building and tuning detections. The **platform baseline** — an Azure subscription, a Log Analytics workspace, Sentinel enabled, least-privilege roles, an ingestion/retention plan, diagnostic-settings capability, and change control — is covered once in `global-requirements-sentinel-solutions.md` (**G1–G7**). Read that first; the items below layer on top of it and are not repeated here.

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing**.

R1. Telemetry is already flowing (a connected workspace)

• **Description:** The detection-driving tables the rules query — e.g. `SigninLogs`, `AuditLogs`, `AzureActivity`, `OfficeActivity`, `SecurityAlert`/`SecurityIncident`, `CommonSecurityLog`, Defender `Device*` — are populated in the workspace.

• **Why required:** A rule can only detect what the workspace ingests. Enabling a rule against an empty table produces a detection that will never fire — a silent blind spot.

• **How to verify:** Sentinel → **Logs** → run `<TableName> | take 10` for each table your target rules need; rows return. Or run `SENTRULE-Discovery.ps1`, which reports which expected tables have recent data.

• **How to implement if missing:** Connect the missing sources first — use the companion *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution. Do not enable rules against tables with no data.

R2. Roles to author rules, watchlists, and read incidents

• **Description: Microsoft Sentinel Contributor** (to create/edit analytics rules and watchlists) for the detection engineer, and at least **Microsoft Sentinel Responder/Reader** for the analysts who will triage the resulting incidents.

• **Why required:** Rule and watchlist authoring is a contributor action; triage needs responder/reader. Scoping keeps change rights small.

• **How to verify:** On the workspace / `RG-Security-Sentinel` → **Access control (IAM)** → the operating accounts hold the required Sentinel roles.

• **How to implement if missing:** Assign the roles at workspace/resource-group scope (prefer PIM-eligible

for Contributor where Entra ID P2 is available — *verify licensing*).

R3. The Content hub solutions for the detections in scope

• **Description:** The Content hub solution packages that carry the **rule templates** you intend to

enable (e.g. the Microsoft Entra ID, Defender XDR, Office 365 solutions each ship detection templates), installed on the workspace.

• **Why required:** Rule templates only appear under **Analytics** → **Rule templates** once their Content

hub solution is installed; you cannot enable a template you have not installed.

• **How to verify:** Sentinel → **Content hub** — the relevant solutions show **Installed**; Sentinel →

Analytics → **Rule templates** lists templates for those products.

• **How to implement if missing:** Content hub → install the solutions matching your connected sources.

Installing adds templates only; it does **not** enable any rule.

R4. A named enrichment data set for watchlists

• **Description:** The reference data that the watchlists will hold: **VIP / high-value users**,

authorized admin source IPs, **terminated users**, and **high-value assets** — as CSVs ready to import into `WL-VIPUsers-Tenant`, `WL-AuthorizedAdminIPs-Tenant`, `WL-TerminatedUsers-Tenant`, and `WL-HighValueAssets-Tenant`.

• **Why required:** Enrichment makes detections precise (fire only for sensitive accounts) and quiet

(suppress known-good sources). Without this reference data, rules cannot key on context and stay noisy.

• **How to verify:** Confirm current CSVs exist and have an owner who keeps them current (a stale

terminated-users list is a real risk).

• **How to implement if missing:** Produce the CSVs from HR/IT sources; assign an owner and a refresh

cadence. They are imported as watchlists in `03-implementation.md`.

R5. An agreed detection priority and false-positive tolerance

• **Description:** A short, written statement from security leadership of **which ATT&CK tactics/techniques

matter most **for your organization and** what false-positive rate is acceptable** before a rule may create incidents (e.g. "≤ X% benign-closed over a Y-day pilot").

• **Why required:** Coverage and tuning are risk decisions, not technical defaults. Without an agreed

priority you cannot rank gaps; without a tolerance you cannot decide when a rule is "tuned enough" to trust.

- **How to verify:** Confirm the written priority/tolerance exists and is owned by security leadership.
- **How to implement if missing:** Facilitate a short session to rank tactics by risk and set a

false-positive tolerance. This is a **leadership determination**; the solution provides the measured coverage and per-rule noise data to inform it, not the decision.

R6. A version-control repository for detection-as-code (optional but recommended)

- **Description:** A Git repository (GitHub or Azure DevOps) to hold exported rules, connected to

Sentinel via **Repositories** (`Repo-Detections-Main`) or fed by the export script.

- **Why required:** Detection-as-code makes rules reviewable, diffable, and portable across

workspaces/tenants, and gives you a rollback point. Without it, tuning history lives only in the portal.

- **How to verify:** Confirm a repo exists and the operating account can push to it; if using Sentinel

Repositories, confirm the connection is configured.

- **How to implement if missing:** Create the repo; either configure Sentinel **Repositories** or use

`SENTRULE-Export.ps1` to export rules to files you commit. *(Sentinel Repositories availability may differ by cloud — verify in your tenant.)*

R7. The PowerShell toolchain

- **Description:** A workstation with the **Az** modules (`Az.Accounts`, `Az.SecurityInsights`,

`Az.OperationalInsights`) and **Microsoft.Graph**, signed in with an MFA-capable admin.

- **Why required:** The five delivered tools use these modules and KQL to discover rules/templates, build

coverage, report false-positive rates, validate, and export detections as code.

- **How to verify:** `Get-Module Az.SecurityInsights,Az.OperationalInsights -ListAvailable` returns

versions; `Connect-AzAccount` succeeds.

- **How to implement if missing:** `Install-Module Az -Scope CurrentUser`; sign in. For Government cloud,

connect with `-Environment AzureUSGovernment` — *verify the correct endpoints for your cloud*.

Global Requirements — Sentinel Solutions

These requirements are common to **every** Sentinel Solutions solution in the standard program model. They are the baseline Azure Monitor / Microsoft Sentinel platform prerequisites that any telemetry, detection, or SIEM engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

Naming convention: every created object is named `<Purpose> - <Scope> - <Stage>` *and reused verbatim throughout the engagement (for example* `Audit Telemetry - Tenant - Pilot`*).*

G1. An Azure subscription

• **Description:** An active Azure subscription in the tenant in scope, with billing enabled.

• **Why:** A Log Analytics workspace and Microsoft Sentinel are Azure resources billed against a subscription; nothing can be provisioned without one.

• **Verify:** Sign in to `https://portal.azure.com`, open `Subscriptions`, and confirm an active subscription resolves and shows status `Active`.

• **Implement-if-missing:** Create or associate an Azure subscription (Enterprise Agreement, CSP, or pay-as-you-go) and confirm billing is in place.

G2. A Log Analytics workspace

• **Description:** A Log Analytics workspace to receive and store ingested telemetry, sized to the data volume and retention in scope.

• **Why:** Microsoft Sentinel runs on top of a Log Analytics workspace; all log data lands here and all queries/analytics run against it.

• **Why region matters:** the workspace region determines data residency and affects egress; pick one deliberately before ingestion starts (it cannot be moved later).

• **Verify:** `portal.azure.com` → `Log Analytics workspaces` → confirm the target workspace exists, note its region and resource group.

• **Implement-if-missing:** Create a workspace named `Security Telemetry - Tenant - Workspace`; choose the region for residency; select the pricing tier that fits expected daily ingestion.

G3. Microsoft Sentinel enabled on the workspace

• **Description:** Microsoft Sentinel added to (enabled on) the Log Analytics workspace in scope.

• **Why:** Connectors, analytics rules, hunting, and incidents are Sentinel features layered on the workspace; without Sentinel enabled you have logs but no SIEM.

• **Verify:** `portal.azure.com` → `Microsoft Sentinel` → confirm the workspace is listed.

• **Implement-if-missing:** `Microsoft Sentinel` → `Create` → select the workspace → `Add`.

G4. Administrative roles (least privilege)

• **Description:** Appropriately scoped roles for the work — Microsoft Sentinel Contributor (or Log Analytics Contributor) on the workspace for configuration, plus the tenant roles needed to set diagnostic settings on the sources (for example a role that can edit Entra ID diagnostic settings).

• **Why:** Each configuration action requires sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** On the workspace/resource group, `Access control (IAM)` → confirm the operating accounts hold the required roles. In Entra → `Roles & admins`, confirm the role that can edit diagnostic settings.

• **Implement-if-missing:** Assign the minimum roles needed at the resource-group or workspace scope; prefer eligible (PIM) activation over standing access where available.

G5. Ingestion and retention cost plan

• **Description:** A documented estimate of daily ingestion (GB/day) and a retention decision (interactive retention vs long-term/archive), with a budget owner.

• **Why:** Sentinel and Log Analytics are billed primarily on data ingested and data retained; identity and device audit logs can be high-volume, so cost must be planned before connectors are switched on.

• **Verify:** Confirm a written estimate exists (sources × expected GB/day) and that retention periods are agreed per table or workspace.

• **Implement-if-missing:** Estimate volume from the source log categories in scope; choose pay-as-you-go or a commitment tier; set workspace/table retention; record the budget and owner.

G6. Diagnostic settings capability on each source

• **Description:** The ability to create diagnostic settings (or enable the relevant data connector) on every source whose telemetry is in scope.

• **Why:** Telemetry only flows to the workspace when a diagnostic setting (or connector) routes the source's log categories to it; this is the switch that turns the pipeline on.

• **Verify:** For each source, confirm a diagnostic setting can be created and that no existing setting already routes the same categories elsewhere (to avoid duplicate ingestion and double billing).

• **Implement-if-missing:** Plan one named diagnostic setting per source that forwards the in-scope categories to the workspace (named per G's convention).

G7. Change control & rollback posture

• **Description:** An approved change process with a scheduled window, plus a documented rollback (which diagnostic settings/connectors/analytics rules were added) before any change.

• **Why:** Enabling connectors and analytics rules changes ingestion volume (and therefore cost) and can generate incidents; controlled changes and a recorded back-out keep the pipeline safe and auditable.

• **Verify:** Confirm an approved change record and a written list of objects to be created (so they can be removed cleanly).

• **Implement-if-missing:** Stand up a lightweight change/approval process; capture the pre-change configuration; document the rollback (delete the named diagnostic setting / disable the connector / disable the rule).

Implementation — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

This is the ordered, do-this-then-that guide. It follows the standard model (**Assess** → **Configure** → **Pilot** → **Validate** → **Expand**) and is **pilot-first**: you map coverage, enable a small high-value rule set, run it in a **low-impact** posture (alerts you triage, incident creation held back) so you can measure noise, **tune** until the false-positive rate is acceptable, validate, then expand across the ATT&CK matrix. Every object is named verbatim so the scripts, testing, and troubleshooting line up. Replace `contoso.com` and the sample user/IP values with your own.

> ***Portal note.*** *Sentinel is reached from* `portal.azure.com` → ***Microsoft Sentinel****, or from the unified* ***Defender portal*** *(*`security.microsoft.com`*) once onboarded. Analytics rules live under* ***Analytics****; templates under* ***Analytics*** → ***Rule templates****; the coverage view under* ***MITRE ATT&CK****; reference data under* ***Watchlists****. Portal paths and the template/NRT sets evolve —* ***verify against your tenant.***
>
> ***Trust gate before incidents.*** *Do* ***not*** *let a new rule create incidents (or drive automation) until it has run in the low-impact pilot posture and its false-positive rate meets the agreed tolerance (****R5****). A rule the SOC does not trust is worse than no rule.*

Object naming (used verbatim throughout)

Object	Name	ATT&CK (example)
VIP/high-value users watchlist	`WL-VIPUsers-Tenant`	—
Authorized admin IPs watchlist	`WL-AuthorizedAdminIPs-Tenant`	—
Terminated users watchlist	`WL-TerminatedUsers-Tenant`	—
High-value assets watchlist	`WL-HighValueAssets-Tenant`	—
Scheduled rule — sign-in brute force	`Rule-BruteForce-Entra-Pilot`	T1110 (Credential Access)
Scheduled rule — atypical travel	`Rule-AtypicalTravel-Entra-Pilot`	T1078 (Defense Evasion / Initial Access)
NRT rule — new privileged role assignment	`Rule-NewAdminRole-Entra-Pilot`	T1098 (Persistence)
Suppression automation rule	`AR-Suppress-KnownScanners-Pilot`	—
MITRE coverage workbook	`Workbook-MITRECoverage-Tenant`	—
Detection-as-code repo connection	`Repo-Detections-Main`	—

(These three rules are worked examples of the map→enable→tune loop; apply the same pattern to the rest of your prioritized gaps.)

Phase 0 — Assess (map coverage, rank gaps)

1. **Confirm the baseline (G1–G7) and R1** — the detection tables the rules need actually have data.
2. **Inventory current rules and templates.** Run `SENTRULE-Discovery.ps1` (read-only): enabled

analytics rules, available rule templates (from installed Content hub solutions), and existing watchlists.

1. **Build the ATT&CK coverage map + gap list.** Run `SENTRULE-Coverage.ps1` — it aggregates the tactics/techniques of enabled rules into a coverage matrix and lists the **uncovered** (gap) cells. Cross-check with Sentinel → **MITRE ATT&CK** (the in-portal coverage view).

1. **Rank gaps by risk against R5.** Order the gaps using leadership's priority (R5). Pick the **top few high-risk gaps** for the pilot — do not try to close everything at once.

• **Gate 0:** you have a measured coverage map, a risk-ranked gap list, and an agreed false-positive tolerance → continue.

Phase 1 — Configure (watchlists, then enable a small high-value set)

Step 1 — Import the enrichment watchlists (R4)

1. Sentinel → **Configuration** → **Watchlists** → **Add new** → create each from its CSV, naming them exactly:

• `WL-VIPUsers-Tenant` (SearchKey = user principal name).

• `WL-AuthorizedAdminIPs-Tenant` (SearchKey = IP/CIDR).

• `WL-TerminatedUsers-Tenant` (SearchKey = user principal name).

• `WL-HighValueAssets-Tenant` (SearchKey = asset/host name).

1. Note each watchlist's alias — rules reference it as `_GetWatchlist('<alias>')`.

Step 2 — Enable the pilot rules against the top-risk gaps

For each pilot rule, enable from a template where one exists, otherwise author a scheduled/NRT rule; in all cases set **entity mappings** and, critically, start in a **low-impact** posture (Step 3).

1. `Rule-BruteForce-Entra-Pilot` **(T1110).** Sentinel → **Analytics** → **Rule templates** → find a sign-in brute-force / failed-logon template (or **Create** → **Scheduled rule**). Query `SigninLogs` for repeated failures per account/IP over a window; **join** `WL-AuthorizedAdminIPs-Tenant` **to exclude sanctioned sources**; map entities **Account**, **IP**, **Host**.

1. `Rule-AtypicalTravel-Entra-Pilot` **(T1078).** Enable the Microsoft "atypical travel" / impossible-

travel template if present, or author a scheduled rule; map **Account**, **IP**. *(Some travel/anomaly detections are Microsoft ML rules — enable rather than author; verify availability.)*

1. `Rule-NewAdminRole-Entra-Pilot` **(T1098).** Sentinel → **Analytics** → **Create** → **NRT rule** → query

`AuditLogs` for privileged-role assignments (e.g. added to a directory role); map **Account** (actor and target). NRT gives low-latency notice of new admin. *(Confirm NRT limits/eligibility in your tenant.)*

Step 3 — Put each pilot rule in the LOW-IMPACT posture

For every rule above, in its **Incident settings**:

1. Set **"Create incidents from alerts triggered by this analytics rule"** to **Disabled** for the

pilot — the rule still generates **alerts** (visible under **Logs**/`SecurityAlert`) you can measure, but it does **not** flood the incident queue or drive automation yet.

1. Leave **alert grouping** configured so related alerts would later group into one incident.
2. Save. This is the analytics-rule equivalent of "report-only": full signal, no operational impact.

Phase 2 — Pilot (let it run, measure the noise)

1. Let the pilot rules run for the agreed pilot window (typically several days).
2. Run `SENTRULE-Report.ps1` — for each pilot rule it reports **alert volume** and, where incidents or analyst dispositions exist, the **false-positive (benign) rate**, plus the top entities driving the alerts.

• **Gate 1:** you have per-rule alert volume and a false-positive signal → proceed to tune.

Phase 3 — Tune (make the alerts trustworthy)

For each noisy pilot rule, apply tuning **in this order** (cheapest, safest first):

1. **Allow-list known-good** — extend the watchlist join (e.g. add the scanner's IP to `WL-AuthorizedAdminIPs-Tenant`) so the rule stops matching sanctioned activity.
1. **Suppress with an automation rule** — create `AR-Suppress-KnownScanners-Pilot` (Sentinel → **Automation** → **Create** → **Automation rule**) to auto-close or suppress incidents matching a known-benign pattern, rather than weakening the detection logic.
1. **Raise the threshold / tighten the window** — increase the count threshold or shorten the lookback so only meaningful bursts fire.
1. **Refine grouping/entities** — group alerts by the right entity so 50 alerts become one incident, not 50.
1. **Re-measure** with `SENTRULE-Report.ps1` after each change. Change **one thing at a time** so you know what worked.

• **Gate 2:** each pilot rule's false-positive rate meets the R5 tolerance → it may be trusted.

Phase 4 — Validate

1. Run `SENTRULE-Validate.ps1` — pass/fail lines aligned to `05-testing.md`: watchlists present and referenced; pilot rules enabled with **entity mappings** set; each pilot rule **still in low-impact posture** until signed off; false-positive rate within tolerance; and the ATT&CK coverage map shows the **targeted gaps now covered**.
1. Re-run `SENTRULE-Coverage.ps1` — confirm the coverage matrix improved on the tactics/techniques you

targeted (the gap cells you picked are now covered).

- **Gate 3 (go/no-go to trust):** all validation lines PASS and coverage improved → promote the rules.

Phase 5 — Expand (promote, export as code, repeat the loop)

1. **Promote the tuned rules.** For each rule that passed Gate 2/3, set **Incident settings → Create incidents **to** Enabled**, and rename the `-Pilot` suffix to your production stage in a change window if you standardize on `-Prod` (keep the convention consistent). Now they create incidents the SOC trusts.

1. **Export as code.** Run `SENTRULE-Export.ps1` to export the enabled rules to ARM/JSON (or feed Sentinel **Repositories** `Repo-Detections-Main`); commit to version control so detections are reviewable and portable.

1. **Deploy the coverage workbook.** Deploy `Workbook-MITRECoverage-Tenant` (start from the Content hub MITRE/coverage workbook) so coverage is visible on an ongoing basis.

1. **Repeat the loop** — take the next batch of high-risk gaps and run Phase 1→4 again. Detection coverage is a program, not a one-time task.

Rollback (for any step)

- **A rule:** disable it (Analytics → the rule → **Disable**) — it stops evaluating immediately. Its exported code remains in the repo.

- **A promotion:** set **Create incidents** back to **Disabled** to return a rule to low-impact posture without deleting it.

- **A watchlist:** delete it; rules that reference it will error until updated — so remove the reference first.

- **Tuning:** because rules are exported as code, revert to the previous committed version to undo a tuning change cleanly.

Testing & Validation — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

Work these gates **in order**. Each is **pass/fail**: do not advance a phase until its gate passes. Run `SENTRULE-Validate.ps1` for a consolidated pass/fail summary, `SENTRULE-Coverage.ps1` for the ATT&CK map, and `SENTRULE-Report.ps1` for per-rule alert and false-positive rates. KQL queries can be run in Sentinel → **Logs**.

Gate 1 — Telemetry the rules need is present

• **Check (KQL):** for each table a target rule queries, `<TableName> | where TimeGenerated > ago(24h) |

count → `non-zero (e.g. SigninLogs, AuditLogs`).

• **Automated:** `SENTRULE-Discovery.ps1` reports which expected tables have recent data.

• **Pass:** every table a pilot rule depends on has recent data. **Fail:** connect the source first

(companion ingestion solution) — do not enable a rule against an empty table.

Gate 2 — Baseline coverage map exists and gaps are ranked

• **Check:** `SENTRULE-Coverage.ps1` produces a tactic/technique coverage matrix and a gap list;

Sentinel → **MITRE ATT&CK** corroborates it.

• **Pass:** a measured coverage map and a risk-ranked gap list (against the R5 priority) exist. **Fail:**

build them before enabling rules — coverage decisions must be measured, not assumed.

Gate 3 — Enrichment watchlists imported and referenced

• **Check:** Sentinel → **Watchlists** — `WL-VIPUsers-Tenant`, `WL-AuthorizedAdminIPs-Tenant`,

`WL-TerminatedUsers-Tenant`, `WL-HighValueAssets-Tenant` exist with rows. A pilot rule query references at least one via `_GetWatchlist('<alias>')`.

• **Automated:** `SENTRULE-Validate.ps1` lists watchlists and flags any pilot rule that should reference

one but does not.

• **Pass:** watchlists present, populated, and referenced. **Fail:** import/repopulate them (Phase 1,

Step 1).

Gate 4 — Pilot rules enabled with entity mappings, in low-impact posture

• **Check:** Sentinel → **Analytics** — `Rule-BruteForce-Entra-Pilot`, `Rule-AtypicalTravel-Entra-Pilot`,

`Rule-NewAdminRole-Entra-Pilot` are **Enabled**, each has **entity mappings** (Account/IP/Host as applicable), and **Create incidents** is **Disabled** (low-impact posture) until sign-off.

• **Automated:** `SENTRULE-Validate.ps1` confirms each pilot rule is enabled, has entity mappings, and

reports its incident-creation setting (flags any that create incidents before Gate 6).

• **Pass:** rules enabled, mapped, and still in low-impact posture. **Fail:** correct the rule (Phase 1,

Steps 2–3).

Gate 5 — Alert volume and false-positive rate measured

• **Check:** `SENTRULE-Report.ps1` reports per pilot rule: alert count over the pilot window and, where

dispositions exist, the benign/false-positive rate.

- **KQL cross-check:**

```
SecurityAlert | where TimeGenerated > ago(7d) | summarize Alerts=count() by AlertName | sort by Alerts desc
```

- **Pass:** you have per-rule alert volume and a false-positive signal. **Fail:** let the pilot run longer / confirm the rule is actually firing.

Gate 6 — False-positive rate within tolerance (trust gate)

- **Check:** For each pilot rule, the benign/false-positive rate meets the agreed **R5** tolerance after tuning (allow-list → suppression → threshold → grouping).
- **Automated:** `SENTRULE-Report.ps1` shows the post-tuning rate per rule against the tolerance.
- **Pass:** every rule to be promoted is within tolerance. **Fail:** keep tuning (Phase 3); do **not** enable incident creation for a rule over tolerance.

Gate 7 — Targeted ATT&CK gaps are now covered

- **Check:** Re-run `SENTRULE-Coverage.ps1` — the specific tactic/technique cells you targeted (e.g. T1110, T1078, T1098) now show as covered by the enabled rules.
- **Pass:** the targeted gap cells moved from uncovered to covered. **Fail:** confirm the rules are tagged with the intended techniques; add the mapping.

Gate 8 — Rules exported as code

- **Check:** `SENTRULE-Export.ps1` produced ARM/JSON (or YAML for Repositories) for the enabled rules, committed to `Repo-Detections-Main` / version control.
- **Pass:** every promoted rule exists as versioned code with a rollback point. **Fail:** export and commit before considering the batch done.

Gate 9 — Promotion and rollback are documented

- **Check:** The change record (**G7**) lists every rule promoted to incident creation, every watchlist, and the automation rule `AR-Suppress-KnownScanners-Pilot`, so each can be disabled/reverted cleanly.
- **Pass:** a named, complete promotion + rollback list exists. **Fail:** complete it before the next expansion loop.

Consolidated pass criteria

The detection set is **tuned, mapped, and trustworthy** when **Gates 1–9 pass**: telemetry present, a measured coverage map with ranked gaps, enrichment watchlists in use, pilot rules enabled and mapped,

alert volume and false-positive rate measured and **within tolerance**, the targeted ATT&CK gaps now covered, rules exported as code, and promotion/rollback documented. Then repeat the loop for the next batch of gaps.

Troubleshooting — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

Common failures, each as **symptom** → **cause** → **fix**. Start with `SENTRULE-Discovery.ps1` (rules, templates, watchlists), `SENTRULE-Coverage.ps1` (the ATT&CK map), and `SENTRULE-Report.ps1` (alert / false-positive rates) — most issues here are visible in one of the three.

An enabled rule never fires

- **Symptom:** A rule is Enabled but produces no alerts, even for activity you can see in the logs.
- **Cause:** The query targets a table with no data (the source is not connected), the KQL returns nothing over the rule's window, the threshold is set too high, or the lookback/frequency is wrong.
- **Fix:** Run the rule's KQL manually in **Logs** over the same window — confirm it returns rows (Gate 1 / R1). Lower the threshold or widen the window to confirm the logic, then re-tighten. Confirm the source table is populated (`<Table> | take 10`).

A rule fires constantly (false-positive flood)

- **Symptom:** One rule dominates the alert/incident queue; analysts close it without reading.
- **Cause:** No allow-list/suppression for known-benign activity, threshold too low, or alerts not grouped so every event becomes its own incident.
- **Fix:** Apply tuning in order (Phase 3): allow-list the known-good source via `WL-AuthorizedAdminIPs-Tenant`; add `AR-Suppress-KnownScanners-Pilot`; raise the threshold; fix grouping. Change one thing at a time and re-measure with `SENTRULE-Report.ps1`.

A rule template does not appear under Rule templates

- **Symptom:** A detection you expected is not in **Analytics** → **Rule templates**.
- **Cause:** The Content hub solution that carries it is not installed (**R3**), or it is a different rule type than expected (e.g. a Microsoft ML rule enabled elsewhere).
- **Fix:** Content hub → install the relevant solution; refresh **Rule templates**. For ML/anomaly detections, check **Analytics** → **Anomalies** / the Microsoft security rules — some are enabled, not authored. *Verify availability in your tenant.*

The MITRE ATT&CK coverage view looks empty or wrong

• **Symptom:** Sentinel → **MITRE ATT&CK** shows little coverage, or `SENTRULE-Coverage.ps1` reports gaps

you thought were covered.

• **Cause:** Enabled rules are **not tagged** with tactics/techniques, or the tags do not match the techniques the rule actually detects.

• **Fix:** Edit each rule → **Set rule logic / MITRE ATT&CK mapping** → tag the correct tactics and techniques. The coverage map is only as accurate as the rule tags. Re-run `SENTRULE-Coverage.ps1`.

A watchlist reference errors in a rule

• **Symptom:** A rule query fails with a watchlist/`_GetWatchlist` error.

• **Cause:** The watchlist alias is wrong, the watchlist was deleted/renamed, or the SearchKey column does not match the join.

• **Fix:** Confirm the alias in **Watchlists**; use the exact alias in `_GetWatchlist('<alias>')`; ensure the join column exists and matches the SearchKey. Do not delete a watchlist a rule references without updating the rule first.

NRT rule will not save / behaves like a scheduled rule

• **Symptom:** `Rule-NewAdminRole-Entra-Pilot` (NRT) cannot be created, or does not run near-real-time.

• **Cause:** NRT has restrictions (limited number of NRT rules, query-shape constraints, supported table set) that the query violates, or NRT is not available in this cloud/tenant.

• **Fix:** Simplify the query to the NRT constraints, or implement it as a frequent **scheduled** rule instead. *Verify current NRT limits/eligibility in your tenant.*

False-positive rate looks fine but real incidents are missed

• **Symptom:** Low noise, but a known test/attack did not raise an incident.

• **Cause:** Over-tuning — the threshold was raised or the suppression widened so far that real activity is now excluded; or the rule is still in low-impact posture (**Create incidents = Disabled**).

• **Fix:** Re-check the incident-creation setting (promote per Phase 5 once trusted); back off the threshold/suppression; run an attack simulation and confirm the alert fires. Tuning is a balance, not a race to zero alerts.

Promoted rule floods incidents after enabling incident creation

• **Symptom:** A rule was quiet in low-impact posture but floods the queue once **Create incidents** is enabled.

- **Cause:** Alert **grouping** was not configured, so each alert opens its own incident.
- **Fix:** Configure alert grouping (group by the key entity over a time window) so related alerts form a single incident; re-measure before re-promoting.

PowerShell / export errors

- **Symptom:** A script errors on connect, on the workspace, or on export.
- **Cause:** Wrong subscription context, missing `Az.SecurityInsights`, insufficient Sentinel role, or (Gov cloud) the wrong environment endpoint.
- **Fix:** `Connect-AzAccount` + `Set-AzContext`; `Install-Module Az.SecurityInsights`; confirm Sentinel Contributor (author/export) or Reader (report/coverage); for Government cloud pass `-Environment AzureUSGovernment` (**R7**) — *verify endpoints for your cloud*.

Best Practices — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

Keep-it-clean guidance for a detection program that stays trusted, measured, and current over time.

Detect against a map, not a checklist

- **Measure coverage; don't assume it.** Enable rules against the **risk-ranked ATT&CK gaps** you measured, not the longest list of templates. A rule that duplicates existing coverage adds noise, not protection.
- **Coverage is depth, not just breadth.** A technique cell showing "covered" means a rule targets it — not that every variant is caught. Prefer a few good detections per high-risk technique over one shallow rule per cell.
- **Re-map after every change.** New connectors, new templates, and retired rules all move the map. Re-run `SENTRULE-Coverage.ps1` on a cadence so the picture stays honest.

Tune for trust — a noisy detection is a liability

- **A rule the SOC ignores is worse than no rule.** It consumes the team and gives false comfort. Treat the **false-positive rate** as a first-class success metric, measured per rule.
- **Tune cheapest-and-safest first:** allow-list known-good (watchlists) → suppress with an automation rule → raise threshold/tighten window → refine grouping/entities. Change **one thing at a time** and re-measure.
- **Suppress, don't blind.** Prefer suppressing a known-benign pattern with `AR-Suppress-KnownScanners-

Pilot` over gutting the detection logic — you keep the detection for the case that matters.

• **Group alerts into meaningful incidents.** Map entities and set grouping so 50 related alerts become one triage-able incident, not 50 tickets.

Pilot before you trust

• **Low-impact posture first.** Run new rules with **incident creation disabled** so they generate measurable alerts without flooding the queue or driving automation. Promote only after the false-positive rate meets the agreed tolerance.

• **Set the tolerance with leadership, not in the tool.** Which techniques matter and what noise is acceptable are **risk decisions (R5)**. The solution measures; leadership decides.

Enrich for precision

• **Keep watchlists current.** A stale `WL-TerminatedUsers-Tenant` misses exactly the risk it exists to catch. Assign an owner and a refresh cadence to every watchlist.

• **Key detections on context.** Use watchlists to fire *only* for VIP/high-value accounts or assets, and to suppress sanctioned admin IPs/scanners — precision beats volume.

Treat detections as code

• **Version every rule.** Export tuned rules with `SENTRULE-Export.ps1` (or Sentinel Repositories) and commit them. Detection logic living only in the portal is unreviewable and unrecoverable.

• **Review and diff changes.** A tuning change is a code change — review it, and keep the previous version as a one-click rollback.

• **Make detections portable.** Detection-as-code lets you promote the same rule from test to production, or across workspaces/tenants, without re-clicking.

Run it as a program

• **Loop, don't launch-and-forget.** Map → enable → pilot → tune → validate → expand, then take the next batch of gaps. Coverage and noise both drift; the loop keeps them honest.

• **Watch the trend.** A falling false-positive rate and a growing covered-technique count are the signs the program is working; track them.

Where to go next

With detections tuned and ATT&CK-mapped, the natural next steps in this catalog are:

• **Sentinel SOAR / automation** — turn the trusted incidents these rules raise into automated response (enrichment, containment, notification) with playbooks and automation rules.

• **Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion** — the foundation this layer depends on; revisit it to add the sources that close coverage gaps you cannot yet detect.

• **Threat hunting / notebooks** — proactive hunting on the same telemetry to find what scheduled rules do not.

• **Microsoft Defender XDR** and **Entra ID** hardening solutions — reduce the attack surface the detections watch.

Deliverables Summary — Microsoft Sentinel Analytics Rules & MITRE ATT&CK Detection Coverage

Everything included in this solution, and what each piece is for.

Documentation

Document	Purpose
Executive Summary (`executive-summary.md`)	One-page business framing: the blind-spots + alert-fatigue problem, the ATT&CK-mapped tuned-detection solution, and the boundary (tunes/maps detections, depends on flowing telemetry, leadership sets priority).
Solution Overview (`01-solution-overview.md`)	What it is, how the technology works (rule types, ATT&CK mapping, tuning, watchlists, automation rules, detection-as-code), the Assess→Configure→Pilot→Validate→Expand model, scope, out-of-scope, and connections used.
Requirements (`02-requirements.md`)	Solution-specific prerequisites (telemetry flowing, authoring/triage roles, Content hub templates, enrichment data, agreed detection priority + false-positive tolerance, a detection-as-code repo, toolchain) — layered on the shared `global-requirements-sentinel-solutions.md` (G1–G7).
Implementation (`03-implementation.md`)	Exact, ordered build: map coverage + rank gaps, import watchlists, enable a small high-value rule set in low-impact posture, pilot + measure, tune to tolerance, validate, then promote/export/repeat — every object named, with rollback.

Document	Purpose
Testing & Validation (`05-testing.md`)	Nine pass/fail gates from telemetry-present and a baseline coverage map through enrichment, low-impact pilot, false-positive-within-tolerance, gap-now-covered, and rules-exported-as-code, referencing the scripts by name.
Troubleshooting (`06-troubleshooting.md`)	The common failure modes (rule never fires, false-positive flood, missing template, empty MITRE view, watchlist errors, NRT limits, over-tuning, incident flood on promotion, PowerShell/export errors) as symptom → cause → fix.
Best Practices (`07-best-practices.md`)	Detect-against-a-map, tune-for-trust, pilot-before-trust, enrich-for-precision, detections-as-code, run-it-as-a-program — plus where to go next in the catalog.

PowerShell tools (5)

All tools are **read-only or export-only**, authenticate **interactively** (no credentials in code), and use the **Az** modules (`Az.Accounts`, `Az.SecurityInsights`, `Az.OperationalInsights`), **Microsoft.Graph** where needed, and **KQL** (via `Invoke-AzOperationalInsightsQuery`) for the reporting. Government-cloud tenants pass the matching environment — *verify endpoints for your cloud*.

Script	Type	What it does
`SENTRULE-Discovery.ps1`	Read-only	Inventories the detection surface: enabled analytics rules (with type and MITRE tags), available rule templates from installed Content hub solutions, existing watchlists, and which expected detection tables have recent data. Changes nothing.
`SENTRULE-Validate.ps1`	Read-only	Runs the `05-testing.md` gates as pass/fail lines: watchlists present + referenced, pilot rules enabled with entity mappings, each pilot rule still in low-impact posture (flags premature incident creation), false-positive rate within tolerance, and targeted gaps covered.

Script	Type	What it does
`SENTRULE-Report.ps1`	Read-only	Per-rule tuning report via KQL: alert volume per rule over the window and, from incident dispositions, the **benign / false-positive rate**, plus the top entities driving each rule — the data that decides when a rule is tuned enough to trust. Exports to CSV.
`SENTRULE-Coverage.ps1`	Read-only	Builds the **MITRE ATT&CK coverage matrix and gap list** from the tactics/techniques of enabled rules, ranks gaps, and exports the matrix to CSV — the map that drives what to enable next.
`SENTRULE-Export.ps1`	Export-only	**Detection-as-code** export: serializes enabled analytics rules to ARM/JSON (and notes the Sentinel Repositories path) for versioning, review, and reuse across workspaces — write-to-file only, never imports/overwrites live rules.

Named objects (verbatim — used across docs, scripts, and tests)

- **Watchlists:** `WL-VIPUsers-Tenant`, `WL-AuthorizedAdminIPs-Tenant`, `WL-TerminatedUsers-Tenant`,

`WL-HighValueAssets-Tenant`

- **Pilot analytics rules:**
- `Rule-BruteForce-Entra-Pilot` — **T1110** Brute Force (Credential Access), scheduled
- `Rule-AtypicalTravel-Entra-Pilot` — **T1078** Valid Accounts, scheduled/ML
- `Rule-NewAdminRole-Entra-Pilot` — **T1098** Account Manipulation (Persistence), NRT
- **Suppression automation rule:** `AR-Suppress-KnownScanners-Pilot`
- **MITRE coverage workbook:** `Workbook-MITRECoverage-Tenant`
- **Detection-as-code repo connection:** `Repo-Detections-Main`

What this solution deliberately does NOT do

- It does **not** connect data sources or size the workspace — that is the companion *Deploy Microsoft Sentinel: Workspace, Data Connectors & Ingestion* solution, which this layer depends on.
- It does **not** let an untrusted rule create incidents — rules pilot in **low-impact posture** and are

promoted only after the false-positive rate meets the agreed tolerance.

- It does **not** decide your **risk tolerance, alerting priorities, or which techniques are in scope** — those are security-leadership determinations; the solution measures coverage/noise to inform them.
- It does **not** claim MITRE ATT&CK coverage is a guarantee — a covered technique means a rule targets it, not that every variant is caught.
- It does **not** invent template, NRT, or ATT&CK-mapping facts — where availability may differ by tenant/cloud it says **verify in your tenant**.

How it is delivered

This solution is delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a formatted Word document and PDF, plus all five PowerShell scripts, in a single ZIP (`SENTRULE-Complete-Solution.zip`) — yours to keep and reuse on every rollout.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SENTRULE-Coverage.ps1

Purpose: Aggregates the MITRE tactics/techniques tagged on enabled analytics rules into a coverage matrix, counts detections per tactic and per technique, and lists the UNCOVERED cells (gaps) so you can rank and close them by risk. Exports the matrix to CSV. Read-only — it changes nothing. The map is only as accurate as the rules' MITRE tags (see 06-troubleshooting.md). HOW TO RUN pwsh -File .\SENTRULE-Coverage.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace"

How to run: pwsh -File .\SENTRULE-Coverage.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace"

SENTRULE-Discovery.ps1

Purpose: Inventories the detection surface so tuning starts from reality: enabled analytics rules (with type and MITRE tactics/techniques where present), available rule templates from installed Content hub solutions, existing watchlists, and which expected detection tables have recent data. Read-only — it changes nothing. HOW TO RUN pwsh -File .\SENTRULE-Discovery.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace"

How to run: pwsh -File .\SENTRULE-Discovery.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace"

SENTRULE-Export.ps1

Purpose: Serializes the workspace's analytics rules to JSON files (one per rule) plus a manifest, so detections become versioned, reviewable, portable code you commit to Repo-Detections-Main. It WRITES FILES ONLY — it never imports, edits, disables, or overwrites any live rule. This is the Gate 8 artifact in 05-testing.md. HOW TO RUN pwsh -File .\SENTRULE-Export.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace" -OutDir .\detections-export pwsh -File .\SENTRULE-Export.ps1 ... -EnabledOnly

How to run: pwsh -File .\SENTRULE-Export.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace" -OutDir .\detections-export pwsh -File .\SENTRULE-Export.ps1 ... -EnabledOnly

SENTRULE-Report.ps1

Purpose: Produces the tuning data that decides when a rule is trustworthy: per-rule ALERT volume over the window (from SecurityAlert) and, from closed incidents' classification, the BENIGN / FALSE-POSITIVE rate per rule (from SecurityIncident). Also lists the top entities driving alerts. This is the signal for Gate 5/6 in 05-testing.md. Read-only — it changes nothing. HOW TO RUN pwsh -File .\SENTRULE-Report.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace" -Days 7 -TolerancePercent 10

How to run: pwsh -File .\SENTRULE-Report.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace" -Days 7 -TolerancePercent 10

SENTRULE-Validate.ps1

Purpose: Validates the detection build as pass/fail lines aligned to 05-testing.md: enrichment watchlists present; pilot rules enabled with entity mappings; each pilot rule still in LOW-IMPACT posture (flags any that create incidents before sign-off); and MITRE tags present so coverage is measurable. Read-only — it changes nothing. (False-positive-within-tolerance is measured by SENTRULE-Report.ps1; coverage-improved by SENTRULE-Coverage.ps1.) HOW TO RUN pwsh -File .\SENTRULE-Validate.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace"

How to run: pwsh -File .\SENTRULE-Validate.ps1 -ResourceGroup RG-Security-Sentinel ` -WorkspaceName "Security Telemetry - Tenant - Workspace"